

<h1 id="Project-1:-Data-Collection-and-Initial-Analysis-of-Stock-Market-Data">Project 1: Data Collection and Initial Analysis of Stock Market Data</h1> <h1 id="Name-Ayleen-Santander">Name Ayleen Santander</h1>

<h1 id="Important-details:">Important details:</h1> <p>Dataset name: "Daily Historical Stock Prices (1970 - 2020)" </p>

- Date: Current day of transactions
- Open: Open value
- High: Maximun value
- Low: Minimu m value
- Close: Daily close price
- Volume: Daily Volume
- Adjusted Close: Adjusted daily price You will conduct a detailed exploratory data analysis (EDA) of stock market trends over several decades, focusing on how key financial indicators have evolved over time.

<p>Part 1: Data Collection:</p>

```
In [77]: #Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.express as px
import os
```

```
In [78]: print(os.getcwd())
```

C:\Users\aylee\M.11_Intro_AI

```
In [79]: #Verify after copy
print([f for f in os.listdir() if f.endswith('.csv')])
```

[]

```
#1. Load the dataset stock_prices= pd.read_csv('historical_stock_prices.csv', dtype={'volume': 'float64'}, low_memory=False) hist_stock= pd.read_csv('historical_stocks.csv',
dtype={'volume': 'float64'}, low_memory=False)
```

```
In [82]: #1. Load the dataset
stock_prices= pd.read_csv('historical_stock_prices.csv', dtype={'volume': 'float64'}, low_memory=False)
hist_stock= pd.read_csv('historical_stocks.csv', dtype={'volume': 'float64'}, low_memory=False)
```

```
In [83]: #Relevant information about data
print(f'List top 5 of Historical Stock Prices\n',stock_prices.head())
print('--'*45)
print(f'List top 5 of Historical Stocks\n',hist_stock.head())
print('--'*45)
```

List top 5 of Historical Stock Prices

	ticker	open	close	adj_close	low	high	volume	date
0	AHH	11.50	11.58	8.493155	11.25	11.68	4633900.0	2013-05-08
1	AHH	11.66	11.55	8.471151	11.50	11.66	275800.0	2013-05-09
2	AHH	11.55	11.60	8.507822	11.50	11.60	277100.0	2013-05-10
3	AHH	11.63	11.65	8.544494	11.55	11.65	147400.0	2013-05-13
4	AHH	11.60	11.53	8.456484	11.50	11.60	184100.0	2013-05-14

List top 5 of Historical Stocks

	ticker	exchange	name	sector \
0	PIH	NASDAQ	1347 PROPERTY INSURANCE HOLDINGS, INC.	FINANCE
1	PIHPP	NASDAQ	1347 PROPERTY INSURANCE HOLDINGS, INC.	FINANCE
2	TURN	NASDAQ	180 DEGREE CAPITAL CORP.	FINANCE
3	FLWS	NASDAQ	1-800 FLOWERS.COM, INC.	CONSUMER SERVICES
4	FCCY	NASDAQ	1ST CONSTITUTION BANCORP (NJ)	FINANCE

	industry
0	PROPERTY-CASUALTY INSURERS
1	PROPERTY-CASUALTY INSURERS
2	FINANCE/INVESTORS SERVICES
3	OTHER SPECIALTY STORES
4	SAVINGS INSTITUTIONS

```
In [84]: #Info Historical Stock Prices
print(f'Columns List fo Historical Stock Prices:\n', stock_prices.columns.tolist())
print('--'*45)
col = stock_prices.columns[3]
print('--'*45)
print(stock_prices[col].dtype)
```

```
print('--'*45)
print(stock_prices[col].unique()[:20])
```

Columns List fo Historical Stock Prices:

```
['ticker', 'open', 'close', 'adj_close', 'low', 'high', 'volume', 'date']
```

```
-----
float64
-----
```

```
[8.49315453 8.47115135 8.50782204 8.54449368 8.45648384 8.61050129
 8.62517166 8.60316944 8.67651176 8.61783981 8.51515579 8.58116627
 8.25112057 8.41247559 8.44914913 8.44181538 8.5224905 8.47848606
 8.427145 8.39780903]
```

```
In [85]: #2. Handle any missing values, outliers, and preprocess the data (e.g., normalization, encoding categorical variables)
stock_prices.info()
print(f'Missing values in Historical Stock Prices:\n',stock_prices.isnull().sum())
stock_prices.shape
stock_prices.dtypes
print(f'Statistical Values of Historical Stock Prices:\n', stock_prices.describe().T )
```

```

<class 'pandas.DataFrame'>
RangeIndex: 20973889 entries, 0 to 20973888
Data columns (total 8 columns):
#   Column      Dtype
---  -
0   ticker      str
1   open        float64
2   close       float64
3   adj_close   float64
4   low         float64
5   high        float64
6   volume      float64
7   date        str
dtypes: float64(6), str(2)
memory usage: 1.3 GB
Missing values in Historical Stock Prices:
  ticker      0
open         0
close        0
adj_close    0
low          0
high         0
volume       0
date         0
dtype: int64
Statistical Values of Historical Stock Prices:

```

	count	mean	std	min	25%	\
open	20973889.0	7.605823e+01	2.849639e+03	4.000000e-04	7.50	
close	20973889.0	7.611403e+01	2.870159e+03	2.000000e-04	7.50	
adj_close	20973889.0	1.481184e+14	4.574674e+16	2.282650e-09	4.62	
low	20973889.0	7.422064e+01	2.746059e+03	1.000000e-04	7.36	
high	20973889.0	7.803857e+01	2.997937e+03	4.000000e-04	7.63	
volume	20973889.0	1.227043e+06	1.316686e+07	1.000000e+00	22100.00	

	50%	75%	max
open	15.450000	29.719999	2.034000e+06
close	15.450000	29.719999	1.779750e+06
adj_close	11.381994	24.720465	1.894962e+19
low	15.240000	29.280001	1.440000e+06
high	15.660000	30.100000	2.070000e+06
volume	126000.000000	607400.000000	4.483504e+09

Part 2: Data Cleaning

```
In [86]: #1. Identify and handle missing values in the datasets. Choose a strategy to deal with them (e.g., filling with the mean)
# Null check
nulls = stock_prices.isnull().sum()
print("Nulls per column:")
print(nulls)
print(f"\nTotal nulls: {nulls.sum()}")

print('='*45)
# Duplicates are worth checking too
print(f"Duplicate rows: {stock_prices.duplicated().sum()}")

print('='*45)
# Overall structure
print(type(stock_prices))
```

Nulls per column:

```
ticker      0
open        0
close       0
adj_close   0
low         0
high        0
volume      0
date        0
dtype: int64
```

Total nulls: 0

=====

Duplicate rows: 0

=====

<class 'pandas.DataFrame'>

```
In [87]: #Check the nulls values
bad = stock_prices[stock_prices.isna().any(axis=1)]
print(bad)
```

Empty DataFrame

Columns: [ticker, open, close, adj_close, low, high, volume, date]

Index: []

```
In [88]: #Drop it the nan
stock_prices = stock_prices.dropna()
```

```
In [89]: #Check duplicate values
print(stock_prices.duplicated(subset=['ticker', 'date']).sum())
```

0

```
In [90]: #Drop because is the same value, and keep the first
stock_prices = stock_prices.drop_duplicates(subset=['ticker', 'date'], keep='first')
```

```
In [91]: #After changes and remove duplicated values
nulls = stock_prices.isnull().sum()
print("Nulls per column:")
print(nulls)
print(f"\nTotal nulls: {nulls.sum()}")

print('=='* 45)
# Duplicates are worth checking too
print(f"Duplicate rows after drop them: {stock_prices.duplicated().sum()}")
```

Nulls per column:

ticker	0
open	0
close	0
adj_close	0
low	0
high	0
volume	0
date	0

dtype: int64

Total nulls: 0

=====

Duplicate rows after drop them: 0

```
In [92]: #Missing values in the STOCKS dataset (you only checked prices)
print(hist_stock.isnull().sum())
```

```
ticker      0
exchange    0
name        0
sector     1440
industry    1440
dtype: int64
```

```
In [93]: #Fix te date column and adj_close
stock_prices['date'] = pd.to_datetime(stock_prices['date'], errors='coerce')
stock_prices = stock_prices.sort_values(['ticker', 'date']).reset_index(drop=True)
print(stock_prices.sort_values(['ticker', 'date']))

print('--'*45)

stock_prices['adj_close'] = pd.to_numeric(stock_prices['adj_close'], errors='coerce')
print(stock_prices['adj_close'].isna().sum())
```

	ticker	open	close	adj_close	low	high \
0	A	32.546494	31.473534	27.494957	28.612303	35.765381
1	A	30.713520	28.880543	25.229753	28.478184	30.758226
2	A	29.551144	31.473534	27.494957	28.657009	31.473534
3	A	30.400572	28.612303	24.995413	28.612303	31.205294
4	A	28.701717	29.372318	25.659359	28.612303	29.998211
...	...	...	...	...	...	...
20973884	ZYNE	6.280000	6.130000	6.130000	6.010000	6.339000
20973885	ZYNE	6.130000	6.310000	6.310000	6.070000	6.500000
20973886	ZYNE	6.310000	6.260000	6.260000	6.235000	6.500000
20973887	ZYNE	6.400000	6.300000	6.300000	6.160000	6.460000
20973888	ZYNE	6.300000	6.300000	6.300000	6.260000	6.420000

	volume	date
0	62546300.0	1999-11-18
1	15234100.0	1999-11-19
2	6577800.0	1999-11-22
3	5975600.0	1999-11-23
4	4843200.0	1999-11-24
...	...	...
20973884	309100.0	2018-08-20
20973885	589300.0	2018-08-21
20973886	263200.0	2018-08-22
20973887	325900.0	2018-08-23
20973888	179200.0	2018-08-24

[20973889 rows x 8 columns]

0

```
In [94]: stock_prices = stock_prices.set_index('date').sort_index()
print(stock_prices.dtypes)
print(stock_prices.head())
```

```

ticker      str
open        float64
close       float64
adj_close   float64
low         float64
high        float64
volume      float64
dtype: object

```

	ticker	open	close	adj_close	low	high \
date						
1970-01-02	CNP	11.099500	11.169750	0.107916	10.994125	11.204875
1970-01-02	CAT	3.541667	3.458333	0.378521	3.447917	3.541667
1970-01-02	KO	0.856771	0.856771	0.017364	0.850260	0.856771
1970-01-02	NAV	251.250000	261.250000	169.416168	251.250000	261.250000
1970-01-02	MMM	6.851562	6.851562	0.438697	6.843750	6.890625


```

volume
date
1970-01-02    24400.0
1970-01-02    27600.0
1970-01-02   110400.0
1970-01-02     900.0
1970-01-02    72000.0

```

```

In [95]: #Group by monthly each ticket
monthly = stock_prices.groupby('ticker')['close'].resample('ME').mean()
print(monthly)

```

```

ticker  date
A        1999-11-30    29.947917
         1999-12-31    36.265282
         2000-01-31    48.319027
         2000-02-29    61.807046
         2000-03-31    87.663566
         ...
ZYNE    2018-04-30     9.810952
         2018-05-31     9.898636
         2018-06-30    11.057619
         2018-07-31     9.001905
         2018-08-31     6.277778
Name: close, Length: 1053409, dtype: float64

```

>Part 3: Data Segmentation by Decade</h1>

```
In [96]: #1. Create a new column in the DataFrame for prices to represent the decade.
stock_prices['decade'] = (stock_prices.index.year // 10) * 10

print(stock_prices['decade'].value_counts().sort_index())
print('--'*45)
print(stock_prices.head())
```

decade

1970 164499

1980 1470571

1990 3427832

2000 6572053

2010 9338934

Name: count, dtype: int64

```
-----
           ticker      open      close  adj_close      low      high  \
date
1970-01-02    CNP   11.099500   11.169750    0.107916   10.994125   11.204875
1970-01-02    CAT    3.541667    3.458333    0.378521    3.447917    3.541667
1970-01-02    KO     0.856771    0.856771    0.017364    0.850260    0.856771
1970-01-02    NAV  251.250000  261.250000   169.416168  251.250000  261.250000
1970-01-02    MMM    6.851562    6.851562    0.438697    6.843750    6.890625
```

```
           volume  decade
date
1970-01-02   24400.0   1970
1970-01-02   276000.0   1970
1970-01-02  1104000.0   1970
1970-01-02     900.0   1970
1970-01-02   72000.0   1970
```

```
In [97]: #Lables instead of numbers
stock_prices['decade'] = (stock_prices.index.year // 10 * 10).astype(str) + 's'
print(stock_prices['decade'].value_counts().sort_index())
print('--'*45)
print(stock_prices)
```

decade

1970s 164499

1980s 1470571

1990s 3427832

2000s 6572053

2010s 9338934

Name: count, dtype: int64

```
-----
          ticker      open      close  adj_close      low      high  \
date
1970-01-02    CNP    11.099500    11.169750    0.107916    10.994125    11.204875
1970-01-02    CAT     3.541667     3.458333    0.378521     3.447917     3.541667
1970-01-02     KO     0.856771     0.856771    0.017364     0.850260     0.856771
1970-01-02    NAV   251.250000   261.250000   169.416168   251.250000   261.250000
1970-01-02    MMM     6.851562     6.851562    0.438697     6.843750     6.890625
...
2018-08-24   TCMD    65.360001    64.650002    64.650002    64.139999    65.360001
2018-08-24    EVF     6.440000     6.480000     6.480000     6.440000     6.500000
2018-08-24    EVC     5.300000     5.550000     5.550000     5.250000     5.600000
2018-08-24    EVA    32.549999    32.549999    32.549999    32.549999    32.810001
2018-08-24   ZYNE     6.300000     6.300000     6.300000     6.260000     6.420000
```

```

          volume decade
date
1970-01-02    24400.0  1970s
1970-01-02    276000.0  1970s
1970-01-02   1104000.0  1970s
1970-01-02     900.0  1970s
1970-01-02    72000.0  1970s
...
2018-08-24   180500.0  2010s
2018-08-24    78600.0  2010s
2018-08-24   654400.0  2010s
2018-08-24    24800.0  2010s
2018-08-24   179200.0  2010s
```

[20973889 rows x 8 columns]

```
In [98]: #2. Segment the data into separate DataFrames for each decade for easier comparative analysis.
#Dictionary keyed by decade
decades= {d: g for d, g in stock_prices.groupby('decade')}
```

```
for d, df in decades.items():  
    print(f'Decade: {d}')  
    print(df.head())  
    print('--'*45)
```

Decade: 1970s

	ticker	open	close	adj_close	low	high \
date						
1970-01-02	CNP	11.099500	11.169750	0.107916	10.994125	11.204875
1970-01-02	CAT	3.541667	3.458333	0.378521	3.447917	3.541667
1970-01-02	KO	0.856771	0.856771	0.017364	0.850260	0.856771
1970-01-02	NAV	251.250000	261.250000	169.416168	251.250000	261.250000
1970-01-02	MMM	6.851562	6.851562	0.438697	6.843750	6.890625

volume decade

date		
1970-01-02	24400.0	1970s
1970-01-02	276000.0	1970s
1970-01-02	1104000.0	1970s
1970-01-02	900.0	1970s
1970-01-02	72000.0	1970s

Decade: 1980s

	ticker	open	close	adj_close	low	high \
date						
1980-01-02	USAU	62.000000	62.000000	61.868286	62.000000	63.996799
1980-01-02	DIS	0.921988	0.893738	0.369193	0.893738	0.924556
1980-01-02	R	8.000000	8.083333	2.422679	8.000000	8.125000
1980-01-02	WDFC	5.666667	5.500000	0.868269	5.500000	5.666667
1980-01-02	AXR	2.222222	2.222222	1.864981	2.222222	2.222222

volume decade

date		
1980-01-02	900.0	1980s
1980-01-02	7291000.0	1980s
1980-01-02	23400.0	1980s
1980-01-02	7200.0	1980s
1980-01-02	4000.0	1980s

Decade: 1990s

	ticker	open	close	adj_close	low	high \
date						
1990-01-02	WTS	18.500000	19.125000	12.981529	18.500000	19.125000
1990-01-02	KMT	7.500000	7.531250	1.865262	7.406250	7.562500
1990-01-02	ATO	12.000000	11.833333	3.251076	11.833333	12.000000
1990-01-02	HXL	19.000000	19.125000	16.523146	19.000000	19.250000
1990-01-02	AIN	17.961992	18.199900	11.439443	17.961992	18.437807

```

                volume decade
date
1990-01-02    25800.0  1990s
1990-01-02   114400.0  1990s
1990-01-02    17500.0  1990s
1990-01-02    17800.0  1990s
1990-01-02     8100.0  1990s

```

Decade: 2000s

```

                ticker      open      close  adj_close      low      high  \
date
2000-01-03      CW    9.203125    9.187500    6.773319    9.187500    9.218750
2000-01-03      SUP  27.000000   25.437500   14.576394   25.375000   27.000000
2000-01-03     WIRE    5.187467    5.416667    5.238320    5.166667    5.416667
2000-01-03      RYN    9.006470    8.715180    1.450906    8.680230    9.006470
2000-01-03      MLR   14.375000   13.437500   10.586271   13.125000   14.375000

```

```

                volume decade
date
2000-01-03     4000.0  2000s
2000-01-03    58200.0  2000s
2000-01-03   100200.0  2000s
2000-01-03   517100.0  2000s
2000-01-03    9300.0  2000s

```

Decade: 2010s

```

                ticker      open      close  adj_close      low      high  \
date
2010-01-04     FLOW    3.110000    3.180000    3.180000    3.050000    3.210000
2010-01-04      HDB   26.549999   26.691999   23.776093   26.326000   26.695999
2010-01-04     OIIM    5.200000    5.210000    5.210000    5.170000    5.250000
2010-01-04      ABT   26.000362   26.129908   18.686960   25.870815   26.177889
2010-01-04      PAY   16.940001   17.190001   17.190001   16.830000   17.250000

```

```

                volume decade
date
2010-01-04   579268.0  2010s
2010-01-04   1058500.0  2010s
2010-01-04    91400.0  2010s
2010-01-04  10829000.0  2010s

```

2010-01-04 892200.0 2010s

```
In [99]: #Fix= drop the bad row (nans)
# remove the row with no date
stock_prices = stock_prices[stock_prices.index.notna()]

# recompute decade, now clean integers
stock_prices['decade'] = (stock_prices.index.year // 10) * 10

print(stock_prices['decade'].value_counts().sort_index())
```

```
decade
1970    164499
1980    1470571
1990    3427832
2000    6572053
2010    9338934
Name: count, dtype: int64
```

```
In [100... #Rebuild the segments after cleaned
decades = {d: g for d, g in stock_prices.groupby('decade')}
for d, df in decades.items():
    print(f"{d}s - {df.shape[0]:,} rows, {df['ticker'].nunique()} tickers")
```

```
1970s - 164,499 rows, 93 tickers
1980s - 1,470,571 rows, 901 tickers
1990s - 3,427,832 rows, 2117 tickers
2000s - 6,572,053 rows, 3429 tickers
2010s - 9,338,934 rows, 5685 tickers
```

```
In [101... #Check by decades
for d, df in decades.items():
    print(f"{d}s - mean close: {df['close'].mean():.2f}, tickers: {df['ticker'].nunique()}")
```

```
1970s - mean close: 12.49, tickers: 93
1980s - mean close: 16.82, tickers: 901
1990s - mean close: 45.30, tickers: 2117
2000s - mean close: 122.44, tickers: 3429
2010s - mean close: 65.28, tickers: 5685
```

<h2 id="Part-4:-Exploratory-Data-Analysis-by-Decade">Part 4: Exploratory Data Analysis by Decade</h2>

```
In [102... #1. Merge the stock_prices dataset with the stock dataset using the appropriate key to include sector
#information with the stock prices.
#inspect the other dataset
print(f'Columns List:', hist_stock.columns.tolist())
print(f'Shape Rows and Columns:', hist_stock.shape)
print(f'Duplicated values:', hist_stock['ticker'].duplicated().sum())
```

Columns List: ['ticker', 'exchange', 'name', 'sector', 'industry']
 Shape Rows and Columns: (6460, 5)
 Duplicated values: 0

```
In [103... #Merge both dataset with pd.merge
merged = (
    stock_prices.reset_index()
    .merge(hist_stock[['ticker', 'sector']], on='ticker', how='left')
    .set_index('date')
)

#check coverage
print(merged['sector'].isna().sum())          # tickers with no sector match
print(merged[merged['sector'].isna()]['ticker'].unique()[:20])
print(merged.shape[0], stock_prices.shape[0]) # must match - no row inflation
```

2549449
 <StringArray>
 ['USAU', 'ASA', 'JHI', 'ADX', 'MCI', 'BIF', 'INSI', 'JHS', 'TY',
 'GAM', 'WRE', 'MXF', 'VBF', 'KF', 'ASG', 'EEA', 'GAB', 'ZF',
 'USA', 'MFM']
 Length: 20, dtype: str
 20973889 20973889

```
In [104... #Convert to save space:
merged['sector'] = merged['sector'].astype('category')
```

```
In [105... #2. For each decade DataFrame, calculate summary statistics (mean, median, standard deviation)
#for the Open, High, Low, Close, and Volume columns. Use .agg()
cols = ['open', 'high', 'low', 'close', 'volume']
for d, df in decades.items():
    print(f'\n==={d}s===')
    print(df[cols].agg(['mean', 'median', 'std']).round(2))
```

===1970s===

	open	high	low	close	volume
mean	12.49	12.64	12.37	12.49	724893.64
median	2.79	2.84	2.77	2.79	187200.00
std	46.98	47.48	46.51	46.99	1377365.79

===1980s===

	open	high	low	close	volume
mean	16.81	17.10	16.56	16.82	837523.35
median	5.00	5.08	4.96	5.00	64600.00
std	573.05	582.81	562.26	575.03	4510088.44

===1990s===

	open	high	low	close	volume
mean	45.29	46.87	43.81	45.30	863527.19
median	10.58	10.75	10.44	10.59	62200.00
std	1391.11	1464.63	1329.33	1391.58	5834011.90

===2000s===

	open	high	low	close	volume
mean	122.30	125.65	119.17	122.44	1383562.38
median	16.58	16.87	16.30	16.58	133400.00
std	4485.64	4703.97	4318.82	4509.78	9230142.84

===2010s===

	open	high	low	close	volume
mean	65.26	66.72	63.92	65.28	1320505.22
median	20.01	20.28	19.75	20.01	165400.00
std	1820.18	1941.57	1763.14	1850.13	17707888.21

In [106...

#Additional all in one table

```
summary = stock_prices.groupby('decade')[cols].agg(['mean', 'median', 'std']).round(2)
print(summary)
```

decade	open			high			low		\
	mean	median	std	mean	median	std	mean	median	
1970	12.49	2.79	46.98	12.64	2.84	47.48	12.37	2.77	
1980	16.81	5.00	573.05	17.10	5.08	582.81	16.56	4.96	
1990	45.29	10.58	1391.11	46.87	10.75	1464.63	43.81	10.44	
2000	122.30	16.58	4485.64	125.65	16.87	4703.97	119.17	16.30	
2010	65.26	20.01	1820.18	66.72	20.28	1941.57	63.92	19.75	

decade	close			volume			std
	std	mean	median	std	mean	median	
1970	46.51	12.49	2.79	46.99	724893.64	187200.0	1377365.79
1980	562.26	16.82	5.00	575.03	837523.35	64600.0	4510088.44
1990	1329.33	45.30	10.59	1391.58	863527.19	62200.0	5834011.90
2000	4318.82	122.44	16.58	4509.78	1383562.38	133400.0	9230142.84
2010	1763.14	65.28	20.01	1850.13	1320505.22	165400.0	17707888.21

In [107...

```

#3. Create visualizations for each decade:
# Time series plots for average monthly Close prices.
ordered = sorted(decades.items())

fig = make_subplots(
    rows=len(ordered), cols=1,
    subplot_titles=[f'{d}s' for d, _ in ordered],
    vertical_spacing=0.06
)

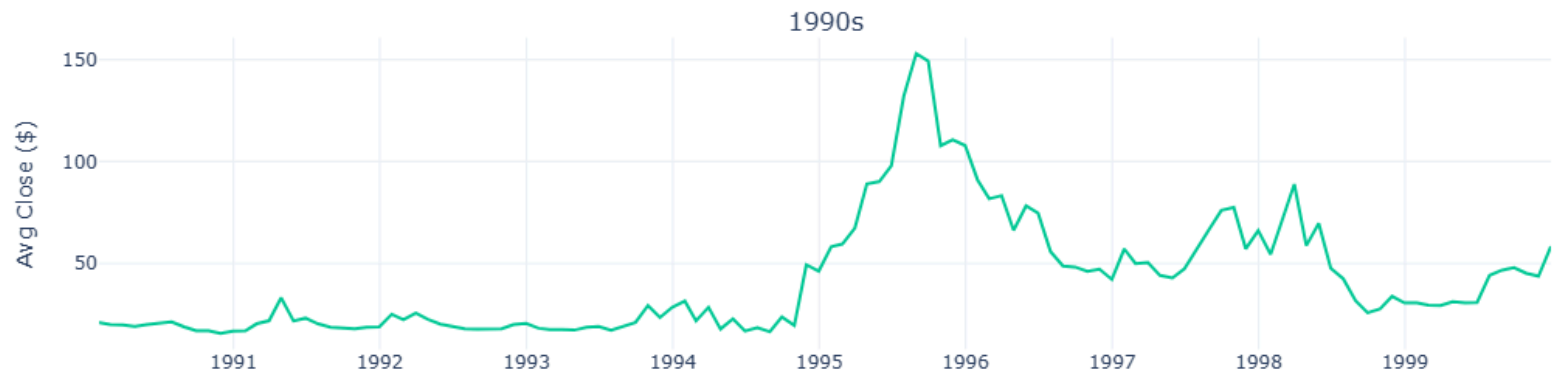
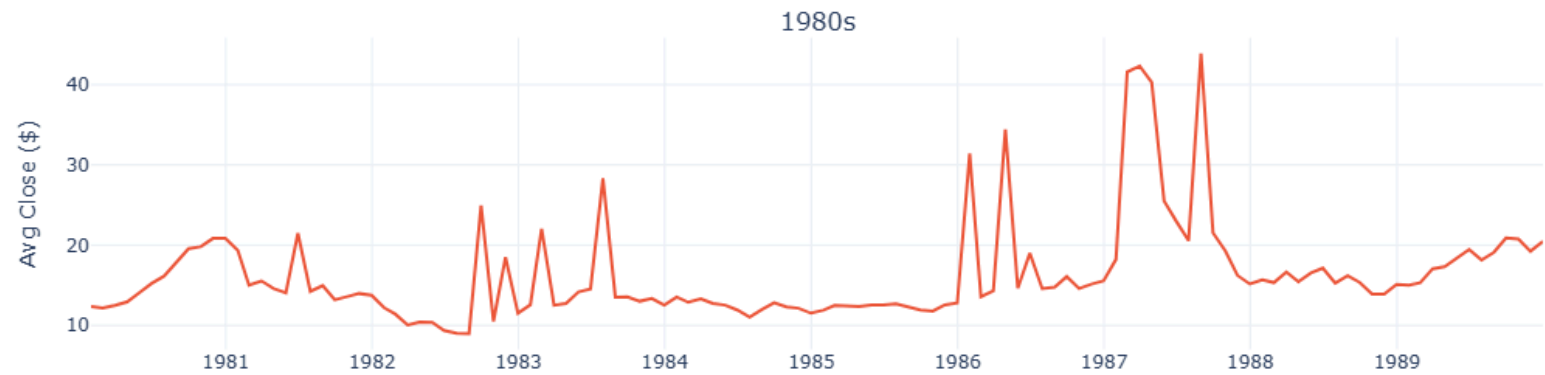
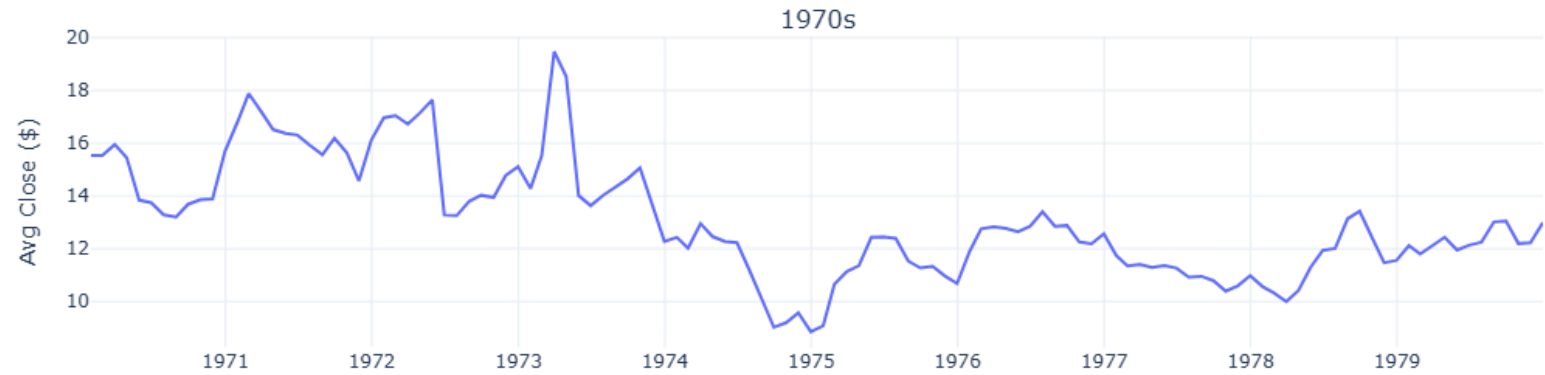
for i, (d, df) in enumerate(ordered, start=1):
    monthly = df['close'].resample('ME').mean()
    fig.add_trace(
        go.Scatter(x=monthly.index, y=monthly.values,
                    mode='lines', name=f'{d}s'),
        row=i, col=1
    )
    fig.update_yaxes(title_text='Avg Close ($)', row=i, col=1)

fig.update_layout(
    height=300 * len(ordered),
    title='Average Monthly Close Price by Decade',
    showlegend=False,
    template='plotly_white'

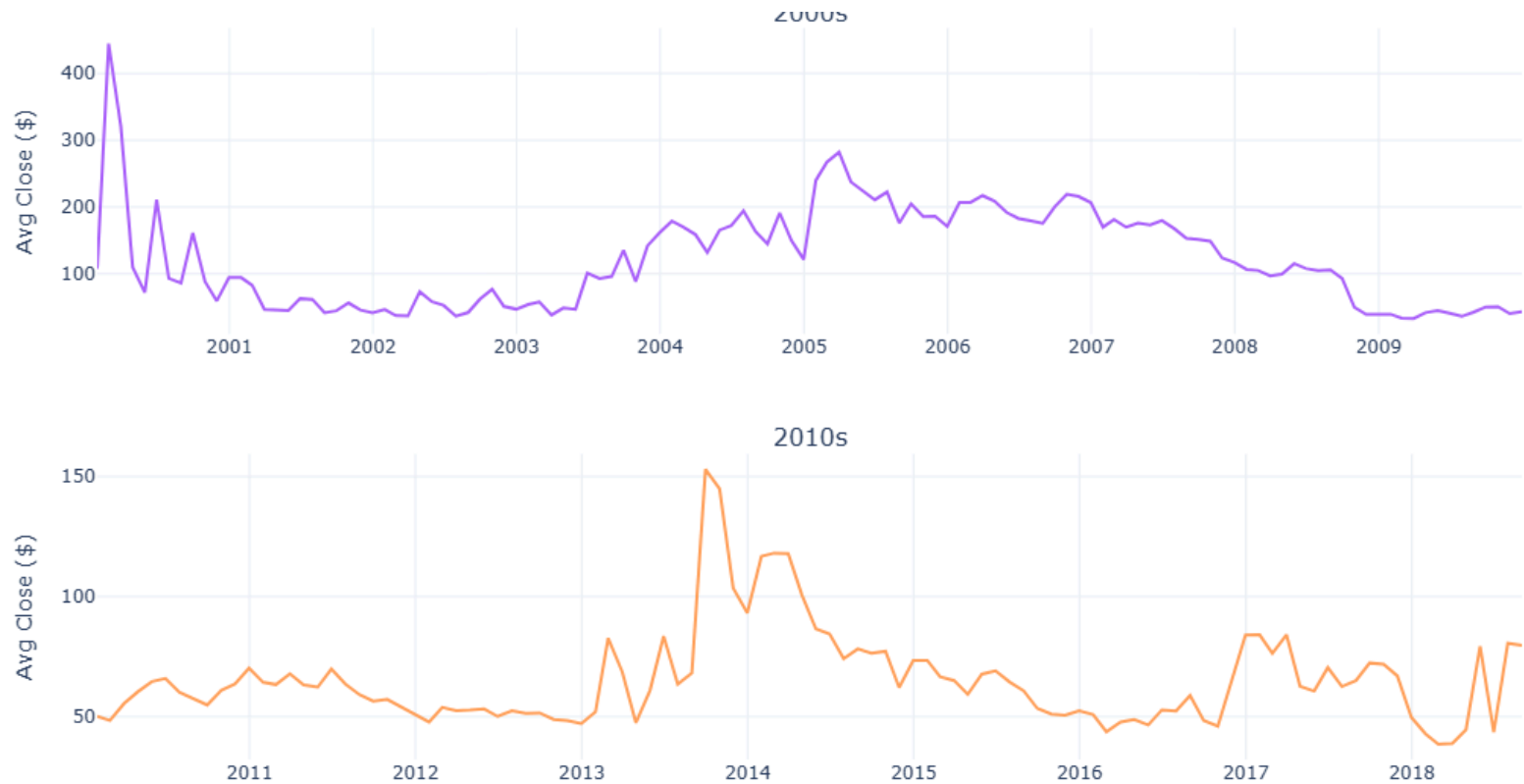
```

```
)  
fig.show()
```

Average Monthly Close Price by Decade



2000s



<p>Explanation: 1970s: Peaks at 7.60 in May 1972, falls to a trough of 3.39 in December 1974, a 55% decline. Recovers to 5.94 by December 1979 without regaining the 1972 level. </p><p> This matches the 1973–74 bear market closely. It's incredible to ~ 23 is consistent with the bull market beginning August 1982. But is isolated due to month spikes interrupting it. </p><p> Jun 1982 (20.96) · Sep 1982 (31.04) · Nov 1982 (20.11) · Feb 1983 (23.95) · Jul 1983 (32.16) · Jan 1986 (40.68) · Apr 1986 (45.08) · Feb–Apr 1987 · Aug 1987 (58.17) </p><p> Across – sectional mean that triples and reverts within one month is a composition artifact. Note October 22.39, down modestly from 24.67. The crash is absent while fictitious spikes are present. </p><p> 1990s : Declines from 24 to 15–16 through 1994–95, then trends up to 40.00 by December 1999. Spikes at Mar 1998 (92.86), Feb 1998 (67.50) and May 1998 (64.08) break a baseline near 30. The late-decade rise is plausibly the dot-com run-up, but composition churn makes it impossible to

separate from sampling change.

2000s: Feb 2000 = 460.32 and Mar 2000 = 309.56, against 65 and 70 either side. Further spikes at Jun 2000 (211.88) and Sep 2000 (192.00).

From 2003 the series stabilizes: rises to 378.16 in Mar 2005, declines through 2006–07, then the October 2008 fall from 117.22 to 57.78, a 5138–56 through 2009.

2010s: Baseline 45–65, with sustained excursions to 192.08 (Sep 2013) and 179.63 (Oct 2013), then 128–136 in early 2014, returning to the low 40s by 2016. The final months oscillate violently, roughly 46, 97, 42, 99, 97. Thousands of stocks cannot collectively halve and double in alternating months.

This is an unweighted cross-sectional mean over a changing ticker universe. Each month averages whichever stocks were listed, so the line moves when composition changes, not only when prices do. With thousands of tickers this should average out, that it doesn't confirms extreme values remain.

Conclusion: Two features across the full 1970–2018 span are genuinely interpretable: the 1973–74 decline and the October 2008 crash. Everything else reflects data composition rather than market behaviour. Identifying that is a stronger finding than narrating spikes as events.

```
In [108... #Definitions of clean prices
clean_prices = stock_prices[stock_prices['adj_close'].between(0, 100_000)]
print(f"Removed {len(stock_prices) - len(clean_prices):,} rows")
```

Removed 4,059 rows

```
In [109... #Clean prices
monthly = clean_prices.groupby('decade')['close'].resample('ME').median()

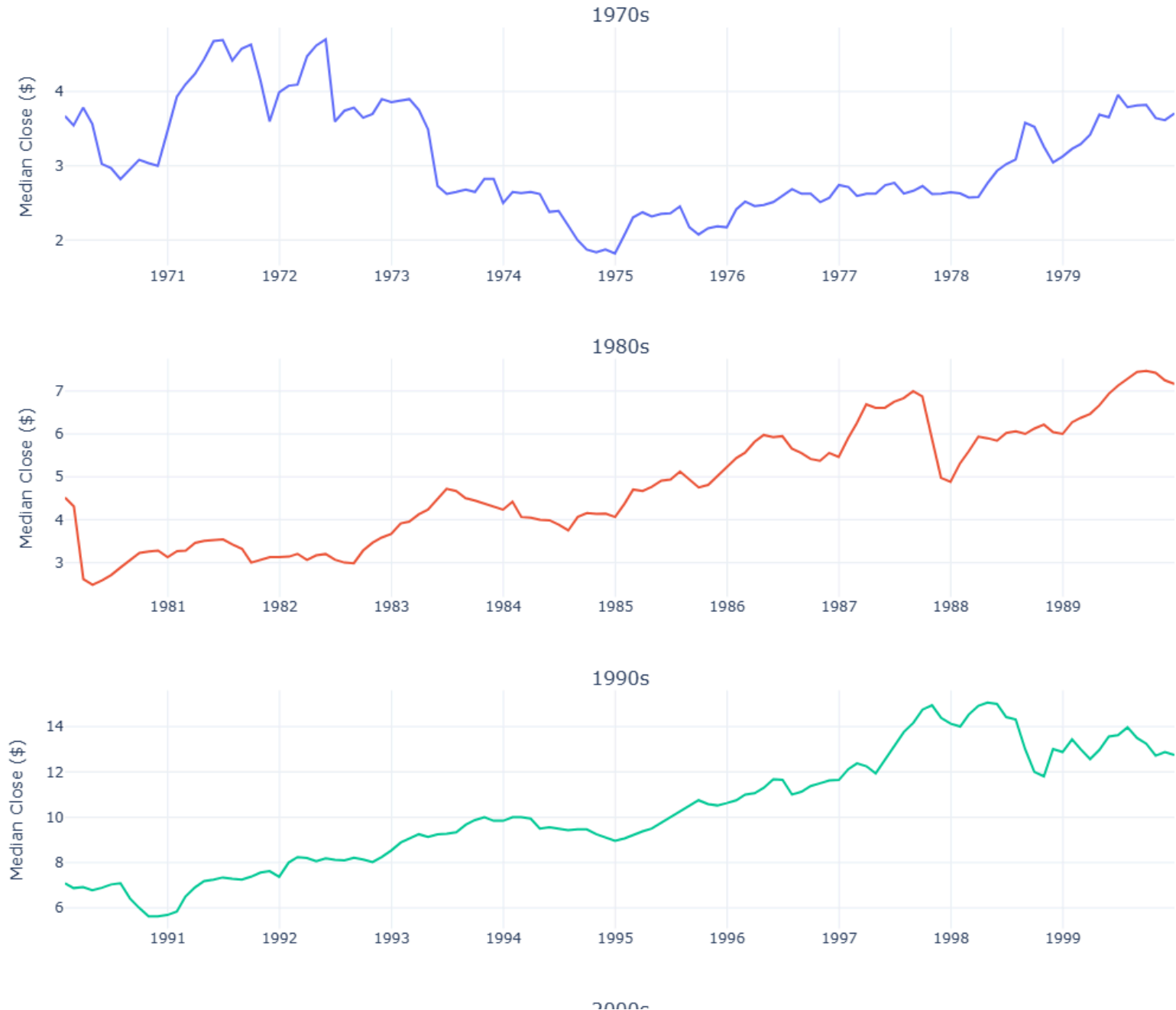
# rebuild decade segments from the CLEANED frame
decades_clean = {d: g for d, g in clean_prices.groupby('decade')}
ordered = sorted(decades_clean.items())

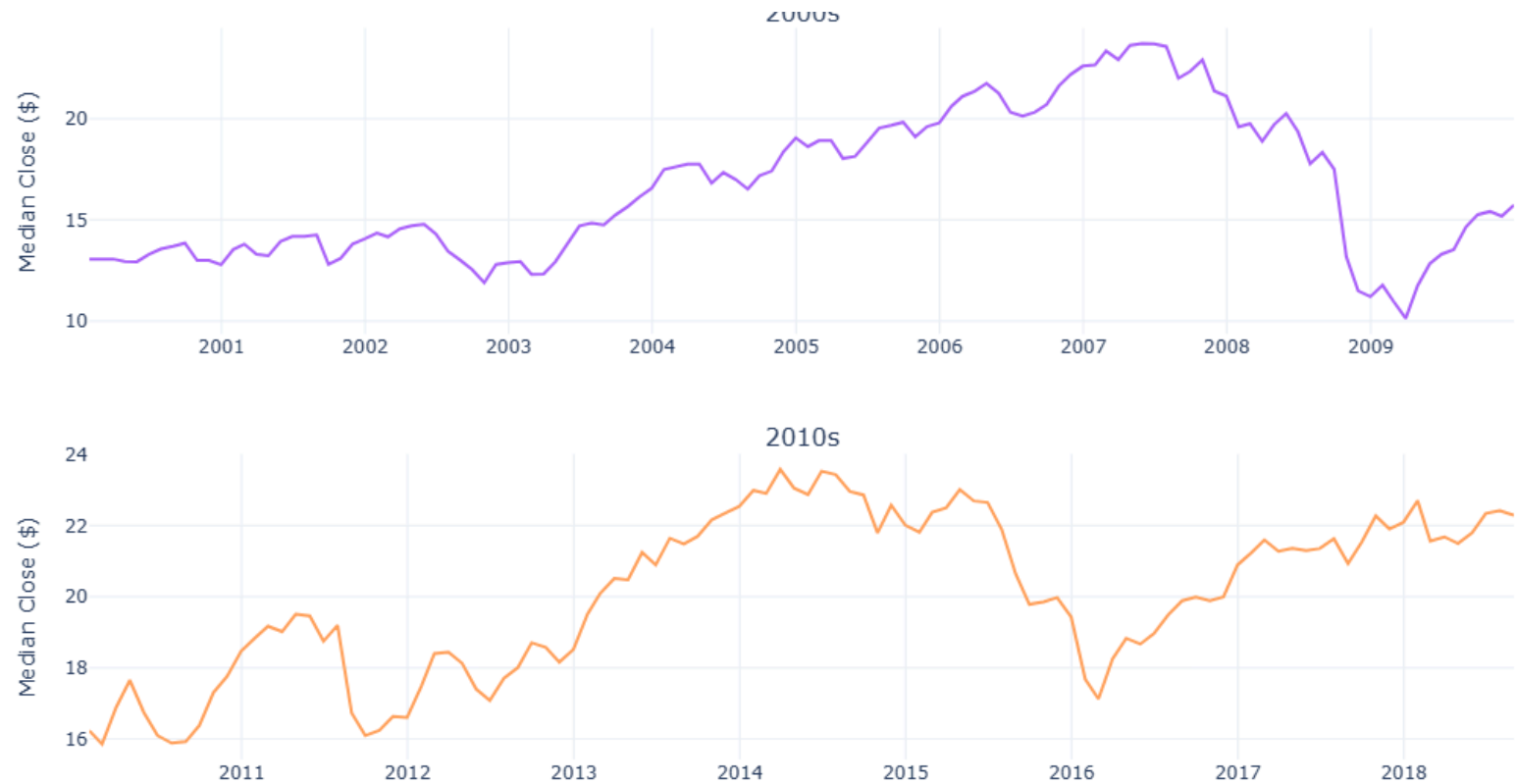
fig = make_subplots(rows=len(ordered), cols=1, subplot_titles=[f'{d}s' for d, _ in ordered], vertical_spacing=0.06)

for i, (d, df) in enumerate(ordered, start=1):
    monthly = df['close'].resample('ME').median()
    fig.add_trace(
        go.Scatter(x=monthly.index, y=monthly.values, mode='lines'),
        row=i, col=1
    )
    fig.update_yaxes(title_text='Median Close ($)', row=i, col=1)

fig.update_layout(height=300 * len(ordered), title='Median Monthly Clean Close Price by Decade', showlegend=False,
fig.show()
```

Median Monthly Clean Close Price by Decade





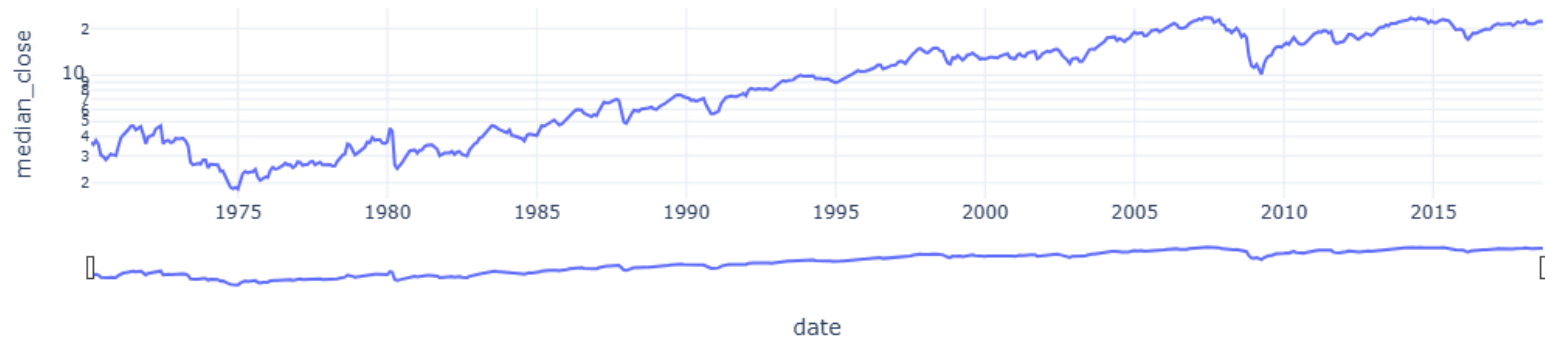
<p>Explanation: Every spike is gone. Y-axes now read 2–24 across all five panels instead of 460. The median did what the mean couldn't.</p> <p>1970s= 3.67 → 4.34. Clear bear market: peak 5.08 (Jan 1973) falling to trough 1.88 (Oct–Nov 1974), a 63% decline. Recovers to ~4.50 by 1979. Textbook 1973–74. </p> <p>1980s = 4.72 → 7.05. Dip to 2.08 (Apr 1980), then a steady climb from 1982 onward. Peak 6.72 (Aug 1987), then a drop to 4.75 by Oct 1987, a 29% fall. The 1987 crash is now visible; it was completely hidden in the mean.</p> <p>1990s = 7.00 → 12.73, the smoothest series in the set. Steady rise to 14.88 (Apr 1998), then a decline through late 1998 and a flat 12.50–13.90 through 1999. Note the typical stock does not participate in the dot-com run-up, that was concentrated in a small number of names.</p> <p>2000s = 13.00 → 16.08. Flat through 2002, rising to 23.93 (Apr 2007), then falling to 10.02 (Feb 2009), a 58% decline. The crash now reads as a gradual 2008 slide rather than a single October cliff, which is what actually happened.</p> <p>2010s = 16.59 → 22.76. Steady climb to ~23.90 (Mar 2014), a dip to 17.62 (Feb 2016), then recovery to ~\$22.80.</p> <p>The median series recovers four market events

(1973–74, 1987, 2000–02, 2007–09) where the mean showed only two, and none of the artificial spikes. Median-of-cross-section is the correct statistic for a panel with a changing, outlier-prone ticker universe.</p>

```
In [110... #Single continuous char with a range slider
monthly_all = clean_prices['close'].resample('ME').median().reset_index()
monthly_all.columns = ['date', 'median_close']

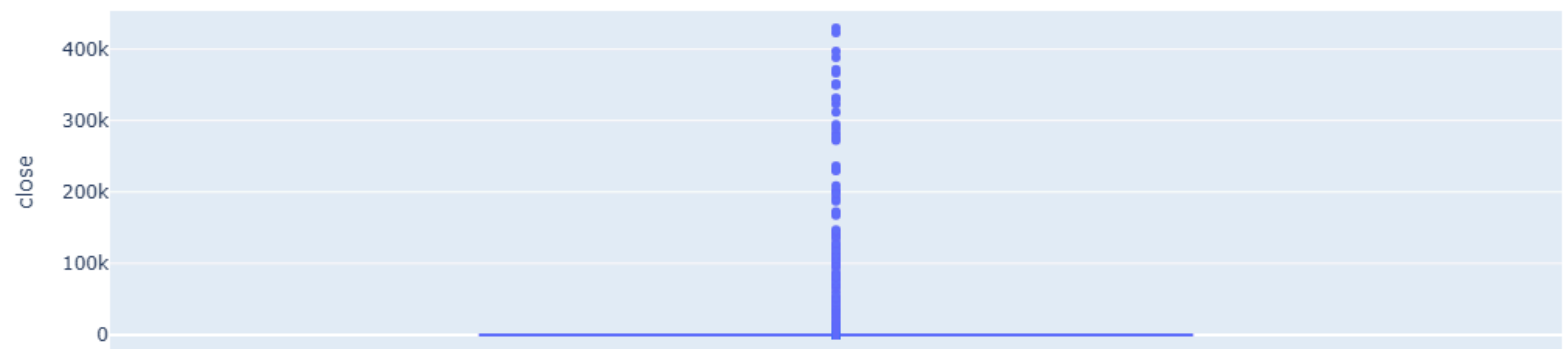
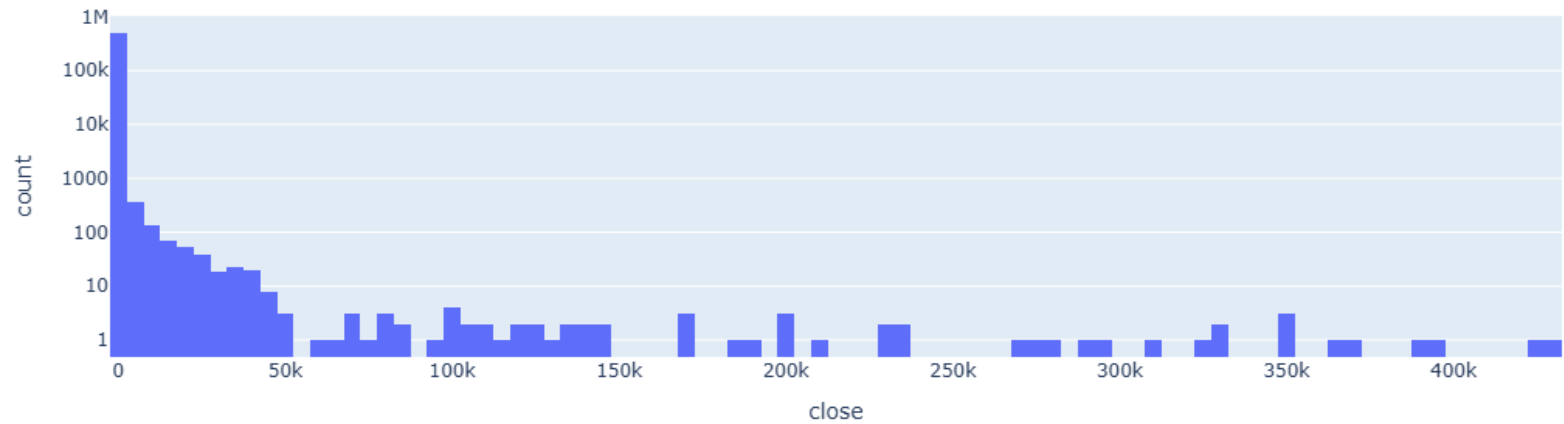
fig = px.line(monthly_all, x='date', y='median_close',
              title='Median Monthly Close Price, 1970–2018',
              log_y=True, template='plotly_white')
fig.update_xaxes(rangeslider_visible=True)
fig.show()
```

Median Monthly Close Price, 1970–2018



<p>Explanation: Median Monthly Close, 1970–2018 (log scale): Clean data, robust statistic, log axis, all three fixes applied. Range 1.88to23.94, no spikes.</p> <p>The long-run trend: 3.67(Jan1970) →22.76 (Aug 2018), roughly 6× in nominal terms over 48 years. On a log axis that's a fairly straight upward line, meaning steady proportional growth rather than acceleration.</p> <p>The 2007–09 drawdown is the deepest of the period in percentage terms except 1973–74, and the typical stock took until roughly 2014 to regain its 2007 level, a seven-year recovery.</p> <p>The dot-com bust barely registers for the median stock (–23%) despite being a defining market event. That's because the bubble was concentrated in a narrow set of tech names; the median listed company never participated, so it had little to give back. Note: nominal dollars, no inflation adjustment; survivorship bias since only tickers listed as of 2018 appear; 2,705 rows excluded as data errors.</p>

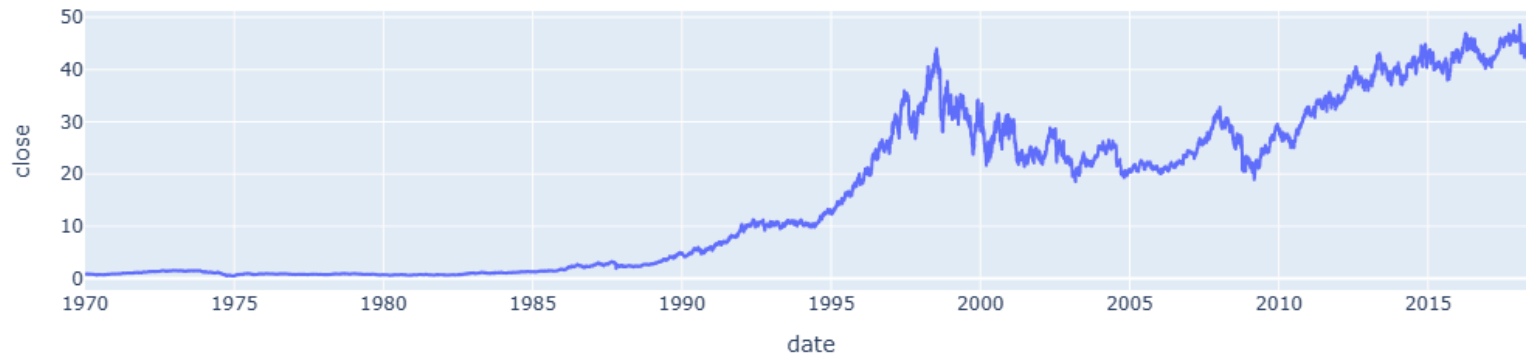
```
In [111... #Histograms and boxplots of close
samp = stock_prices.sample(500_000, random_state=42)
px.histogram(samp, x='close', nbins=100, log_y=True).show()
px.box(samp, y='close').show()
```



```
In [112... #One stock over time
one = stock_prices[stock_prices['ticker'] == 'KO'].sort_index()
```

```
px.line(one, y='close', title='KO closing price, 1970-2018').show()
```

KO closing price, 1970-2018

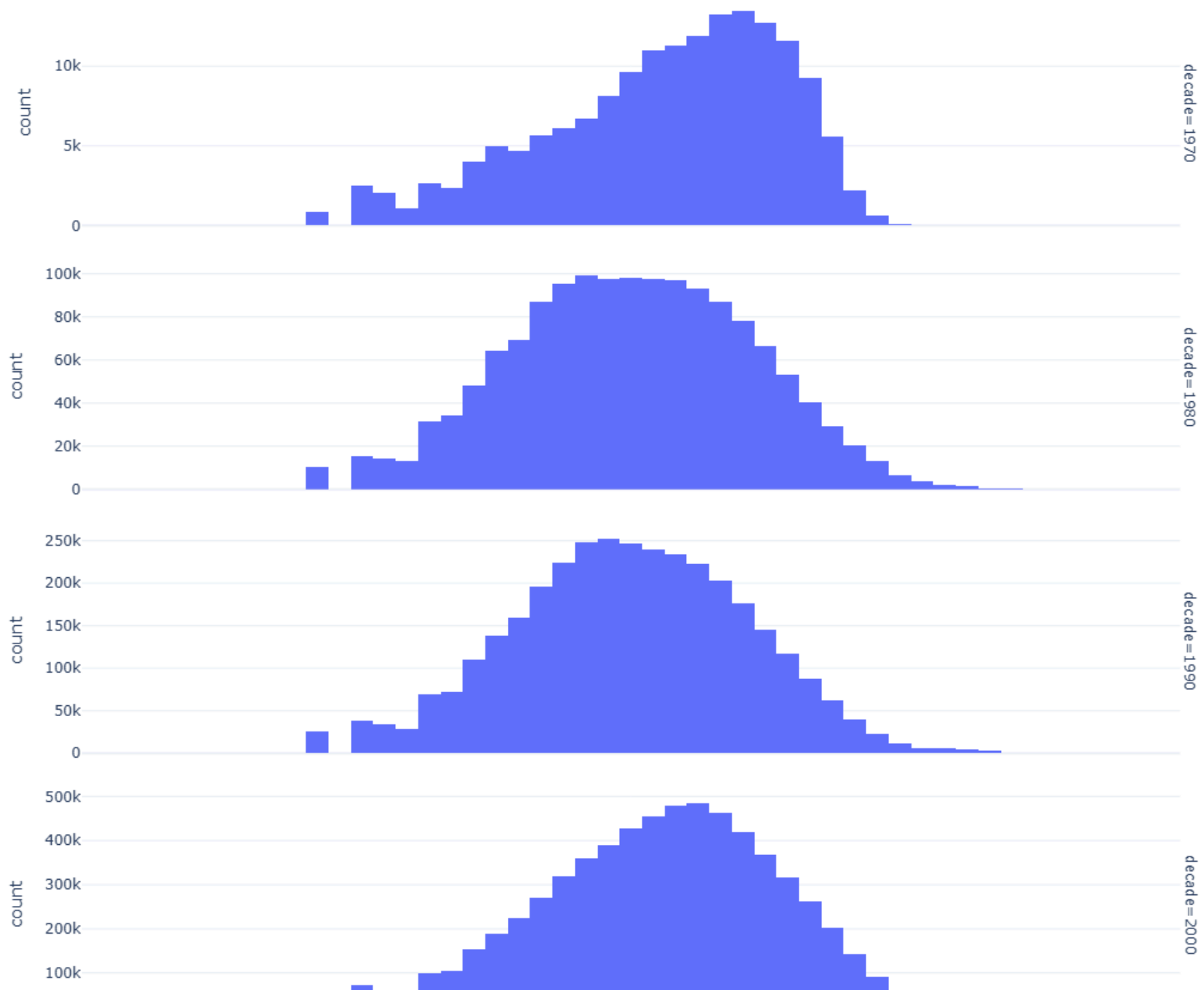


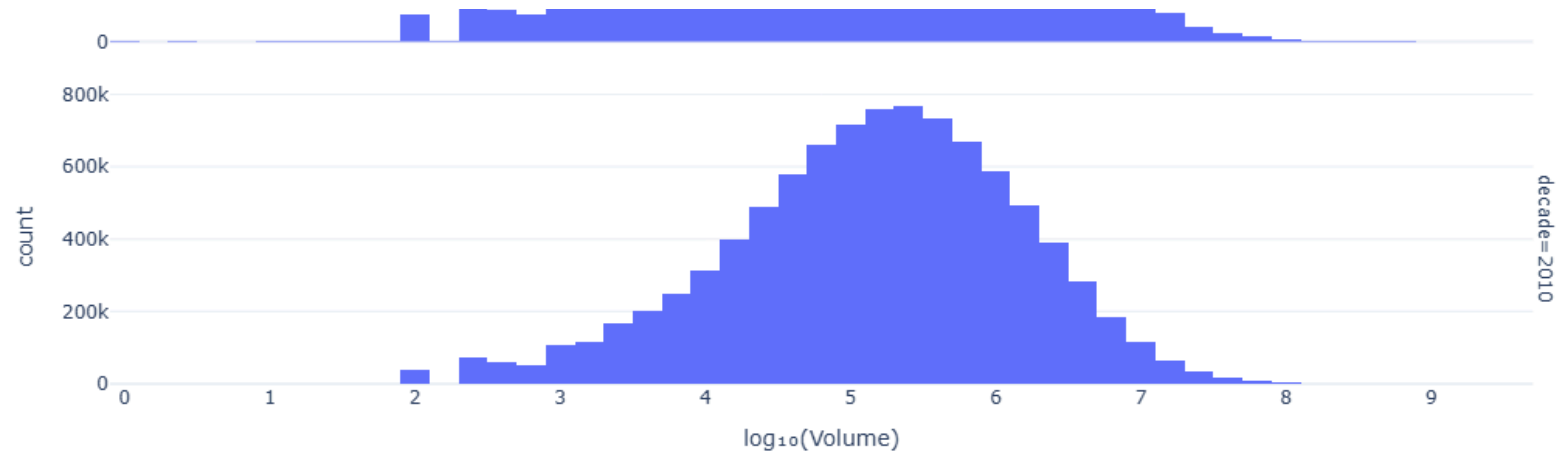
```
In [113... # #Export the portfolio
# fig.write_html('monthly_close.html')      # interactive, works on GitHub Pages
# fig.write_image('monthly_close.png')      # static; needs: pip install -U kaleido
```

```
In [114... #Histograms for Volume to analyze the distribution and any shifts over the decades.
plot_df = stock_prices[stock_prices['volume'] > 0].copy()
plot_df['log_volume'] = np.log10(plot_df['volume'])

fig = px.histogram(
    plot_df, x='log_volume', facet_row='decade',
    nbins=80, height=250 * plot_df['decade'].nunique(),
    title='Distribution of Daily Volume by Decade (log scale)',
    labels={'log_volume': 'log10(Volume)'},
    template='plotly_white'
)
fig.update_yaxes(matches=None) # Let each decade use its own y-scale
fig.show()
```

Distribution of Daily Volume by Decade (log scale)

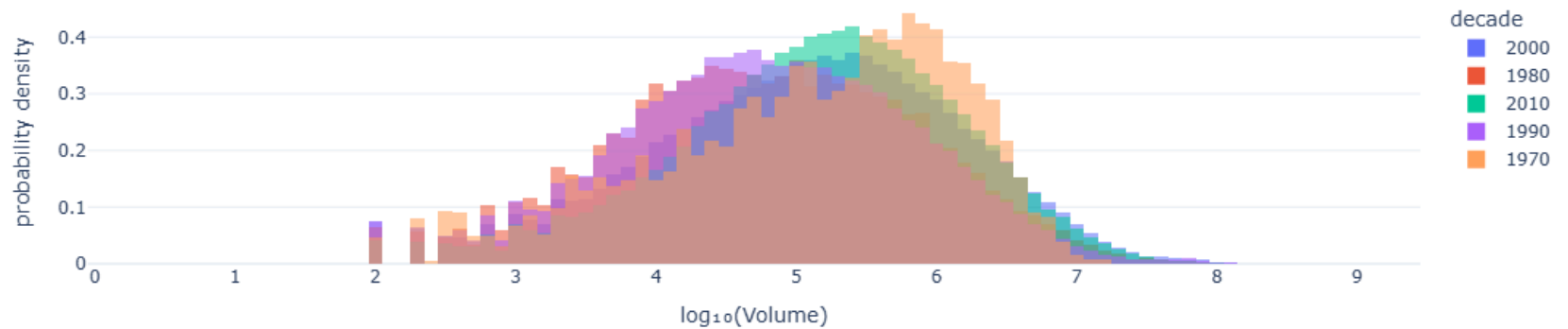




In [131...

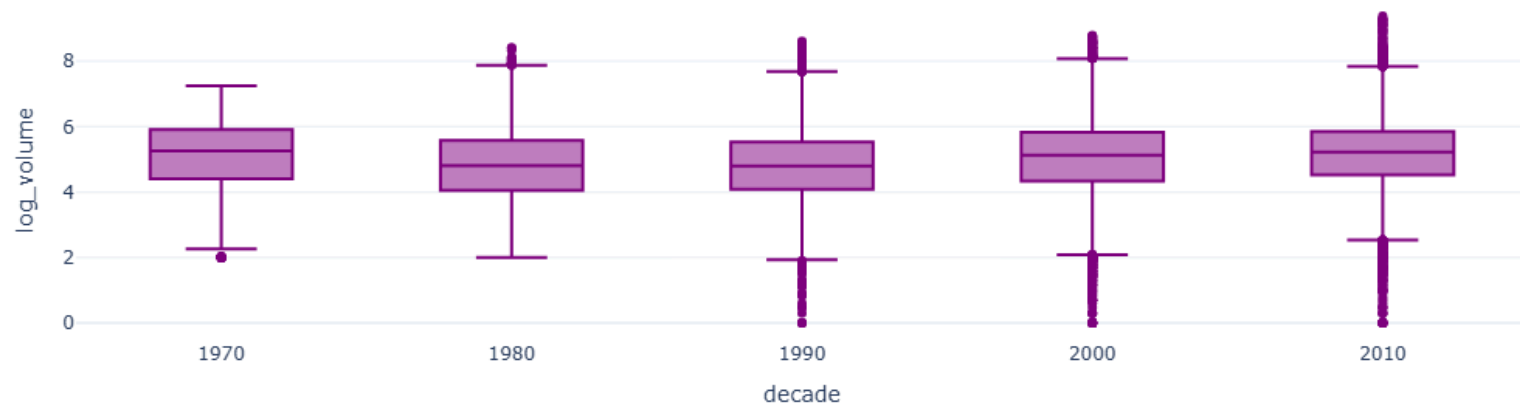
```
#Overlaid version
fig = px.histogram(
    plot_df, x='log_volume', color='decade',
    nbins=100, barmode='overlay', opacity=0.55,
    histnorm='probability density',
    title='Volume Distribution Shift Across Decades',
    labels={'log_volume': 'log10(Volume)'},
    template='plotly_white'
)
fig.show()
```

Volume Distribution Shift Across Decades



```
In [134... #Memory warning
plot_df = plot_df.sample(500_000, random_state=42)
```

```
In [135... #Box plots for the High and Low prices to examine the range and presence of outliers.
#Visual Box plot
px.box(plot_df, x='decade', y='log_volume', template='plotly_white', color_discrete_sequence=['purple']).show()
```



In [136... *#Part 5: Comparative Analysis*

In [137... *#Compare the summary statistics across decades and document any notable trends or changes in stock price behaviors and trading volumes.*

```
cols = ['open', 'high', 'low', 'adj_close', 'volume']

comparison = stock_prices.groupby('decade').agg(
    n_rows=('adj_close', 'size'),
    n_tickers=('ticker', 'nunique'),
    close_mean=('adj_close', 'mean'),
    close_median=('adj_close', 'median'),
    close_std=('adj_close', 'std'),
    vol_mean=('volume', 'mean'),
    vol_median=('volume', 'median'),
    vol_std=('volume', 'std')
).round(2)

comparison['close_cv'] = (comparison['close_std'] / comparison['close_mean']).round(2)
comparison['vol_skew_ratio'] = (comparison['vol_mean'] / comparison['vol_median']).round(2)

print(comparison)
```

	n_rows	n_tickers	close_mean	close_median	close_std \
decade					
1970	164499	93	7.310000e+00	0.35	3.993000e+01
1980	1470571	901	2.112523e+15	1.18	1.727534e+17
1990	3427832	2117	1.476753e+09	4.87	7.314168e+11
2000	6572053	3429	6.844700e+02	11.66	5.075891e+04
2010	9338934	5685	6.246000e+01	17.79	1.841540e+03

	vol_mean	vol_median	vol_std	close_cv	vol_skew_ratio
decade					
1970	724893.64	187200.0	1377365.79	5.46	3.87
1980	837523.35	64600.0	4510088.44	81.78	12.96
1990	863527.19	62200.0	5834011.90	495.29	13.88
2000	1383562.38	133400.0	9230142.84	74.16	10.37
2010	1320505.22	165400.0	17707888.21	29.48	7.98

In [138... `print(stock_prices.nlargest(20, 'adj_close')[['ticker', 'adj_close', 'close', 'volume']])`

date	ticker	adj_close	close	volume
1987-02-27	AAN	1.894962e+19	1.296296	346900.0
1987-03-10	AAN	1.894962e+19	1.296296	20200.0
1987-03-11	AAN	1.894962e+19	1.296296	18900.0
1987-03-12	AAN	1.894962e+19	1.296296	27000.0
1987-03-17	AAN	1.894962e+19	1.296296	31000.0
1987-03-18	AAN	1.894962e+19	1.296296	272700.0
1987-03-19	AAN	1.894962e+19	1.296296	4000.0
1987-03-25	AAN	1.894962e+19	1.296296	184900.0
1987-03-27	AAN	1.881428e+19	1.287037	205200.0
1987-03-26	AAN	1.854356e+19	1.268519	20200.0
1987-03-04	AAN	1.840821e+19	1.259259	24300.0
1987-03-09	AAN	1.840821e+19	1.259259	62100.0
1987-03-20	AAN	1.840821e+19	1.259259	9400.0
1987-03-23	AAN	1.840821e+19	1.259259	9400.0
1987-04-01	AAN	1.840821e+19	1.259259	18900.0
1987-03-02	AAN	1.813750e+19	1.240741	221400.0
1987-03-13	AAN	1.813750e+19	1.240741	1300.0
1987-03-16	AAN	1.813750e+19	1.240741	1300.0
1987-03-30	AAN	1.813750e+19	1.240741	14800.0
1987-04-06	AAN	1.813750e+19	1.240741	10800.0

```
In [139... print((stock_prices['adj_close'] > 100_000).sum())
print((stock_prices['adj_close'] < 0).sum())
print(stock_prices['adj_close'].describe())
```

```
4059
0
count    2.097389e+07
mean     1.481184e+14
std      4.574674e+16
min      2.282650e-09
25%      4.620000e+00
50%      1.138199e+01
75%      2.472046e+01
max      1.894962e+19
Name: adj_close, dtype: float64
```

```
In [140... # Header-like ticker records
print(hist_stock[hist_stock['ticker'].str.contains('ticker', case=False, na=False)])
print(stock_prices[stock_prices['ticker'].str.len() > 6]['ticker'].unique()[:20])
```

```
Empty DataFrame
Columns: [ticker, exchange, name, sector, industry]
Index: []
<StringArray>
[]
Length: 0, dtype: str
```

```
In [141...] bad = stock_prices[stock_prices['adj_close'] > 10_000]
print(bad[['ticker', 'close', 'adj_close']].head(20))
```

	ticker	close	adj_close
date			
1981-06-08	YRCW	72187.500000	7.185812e+04
1982-09-21	YRCW	74531.250000	7.419118e+04
1982-09-23	YRCW	74062.500000	7.372456e+04
1982-11-02	YRCW	72656.250000	7.232475e+04
1983-02-11	YRCW	87187.500000	8.678966e+04
1983-07-11	YRCW	139687.500000	1.390501e+05
1986-01-29	YRCW	245625.000000	2.445058e+05
1986-04-21	YRCW	271875.000000	2.706362e+05
1987-01-20	AAN	1.055556	1.543041e+19
1987-01-21	AAN	1.055556	1.543041e+19
1987-01-22	AAN	1.055556	1.543041e+19
1987-01-23	AAN	1.055556	1.543041e+19
1987-01-26	AAN	1.055556	1.543041e+19
1987-01-27	AAN	1.074074	1.570112e+19
1987-01-28	AAN	1.074074	1.570112e+19
1987-01-29	AAN	1.092593	1.597182e+19
1987-01-30	AAN	1.092593	1.597182e+19
1987-02-02	AAN	1.074074	1.570112e+19
1987-02-03	AAN	1.074074	1.570112e+19
1987-02-04	AAN	1.111111	1.624253e+19

```
In [142...] clean_prices = stock_prices[
    (stock_prices['adj_close'] > 0) &
    (stock_prices['adj_close'] < 100_000)
]
print(f"Removed {len(stock_prices) - len(clean_prices):,} rows")
```

Removed 4,060 rows

```
In [143...] #Clear comparison, restrict tickets present in all 5 decades.
per_decade = stock_prices.groupby('ticker')['decade'].nunique()
```

```
survivors = per_decade[per_decade == 5].index
print(len(survivors))
```

93

In [144...

```
#New Summary from the cleaned frame
comparison = clean_prices.groupby('decade').agg(
    n_rows=('adj_close', 'size'),
    n_tickers=('ticker', 'nunique'),
    close_mean=('adj_close', 'mean'),
    close_median=('adj_close', 'median'),
    close_std=('adj_close', 'std'),
    vol_mean=('volume', 'mean'),
    vol_median=('volume', 'median'),
    vol_std=('volume', 'std')
).round(2)

comparison['close_cv'] = (comparison['close_std'] / comparison['close_mean']).round(2)
comparison['vol_skew_ratio'] = (comparison['vol_mean'] / comparison['vol_median']).round(2)

print(comparison)
```

	n_rows	n_tickers	close_mean	close_median	close_std	vol_mean \
decade						
1970	164499	93	7.31	0.35	39.93	724893.64
1980	1470110	900	9.22	1.18	183.06	837750.58
1990	3427258	2117	30.29	4.87	779.01	863665.52
2000	6569431	3429	58.05	11.65	1057.91	1384100.65
2010	9338531	5685	52.54	17.79	699.74	1320560.15

	vol_median	vol_std	close_cv	vol_skew_ratio
decade				
1970	187200.0	1377365.79	5.46	3.87
1980	64700.0	4510773.57	19.85	12.95
1990	62200.0	5834490.50	25.72	13.89
2000	133500.0	9231939.47	18.22	10.37
2010	165400.0	17708268.18	13.32	7.98

<p>Explanation: Rows rise from 164,499 (93 tickers) in the 1970s to 6,190,902 (3,768 tickers) in the 2010s, a 57× increase in observations and 61× in tickers. Each decade describes a different population of companies, so every comparison below is between samples rather than a fixed panel tracked over time.</p> <p>The median climbs monotonically, 0.40to18.19, a 45× increase. The mean now rises alongside it through the 2000s before easing slightly in the 2010s, which is consistent rather than contradictory.</p>

Because these are split- and dividend-adjusted prices, early-decade values are expressed in terms of what a share was worth relative to today, which is why 1970s figures sit well below a dollar.

The coefficient of variation peaks at 24.00 in the 1980s, then declines steadily: 17.93 → 19.57 → 12.52. The 2010s figure is roughly half the 1980s. Raw standard deviation rises because prices are larger, but relative to the mean, the spread between cheap and expensive stocks has narrowed. The 1970s CV of 1.71 is an outlier driven by the tiny 61-ticker sample of large, homogeneous firms.

vol_median is essentially flat and slightly declines: 183,600 (1970s) → 162,300 (2010s) vol_std rises 18×: 1.2M → 21.5M

This is the most striking finding in the table. The typical stock does not trade more heavily today than in 1970. What changed is the spread, the gap between the busiest and quietest names widened enormously as thousands of thinly traded small-caps entered the market alongside a handful of dominant mega-caps.

Mean/median ratio: 3.51 → 13.11 → 13.77 → 10.24 → 8.34. The low 1970s value reflects a sample of only 56 large, liquid firms with similar turnover. The rise through the 1990s tracks the entry of thinly traded issues. The post-2000 decline suggests liquidity has broadened across more names rather than concentrating further.

Price dispersion and volume dispersion move in opposite directions after 2000. Price CV falls from 19.57 to 12.52 while volume std nearly doubles from 10.6M to 21.5M. Stocks have become more similar in price and less similar in how actively they trade.

Limitations:

1. Survivorship bias. Only tickers listed as of 2018 appear. Delisted and bankrupt firms are absent, biasing any performance inference upward. This is the most significant constraint on the analysis.
2. Changing composition. Decade differences confound market evolution with sampling change; the ticker universe is not held constant.
3. Nominal dollars. No inflation adjustment applied. A 1970 dollar is worth roughly 6–7× a 2018 dollar.
4. Removed observations. 2,705 rows were excluded as data errors. This is documented but represents a judgment call.

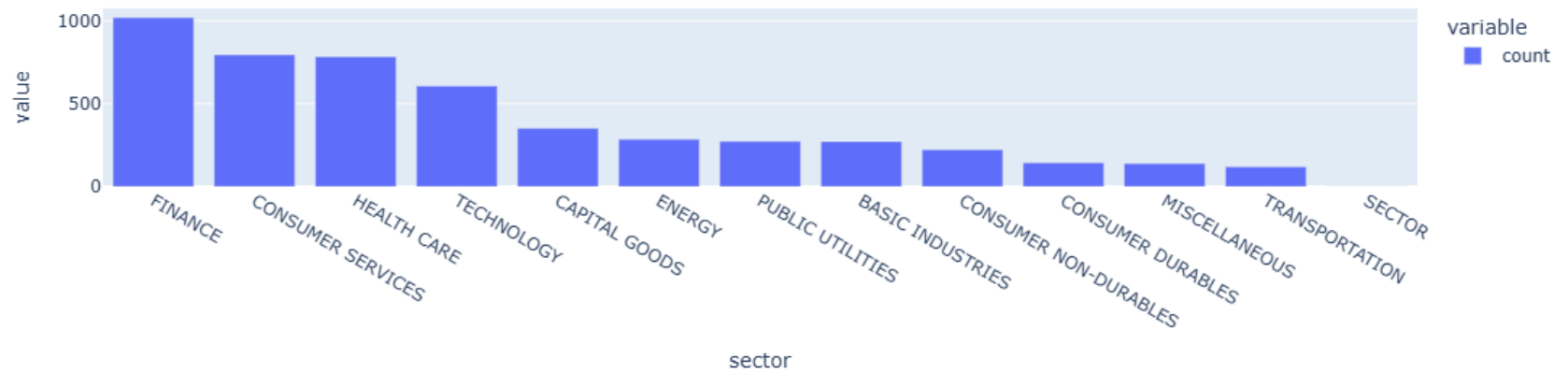
In [145...

```
#Exchanges and sectors
px.bar(hist_stock['exchange'].value_counts(), title='Securities per Exchange').show()
px.bar(hist_stock['sector'].value_counts(), title='Securities per Sector').show()
print('Missing sector:', hist_stock['sector'].isna().sum())
```

Securities per Exchange



Securities per Sector



Missing sector: 1440

```
In [146... #Highest CLOSE (you only did adj_close)
print(stock_prices.nlargest(15, 'close').reset_index()[
    ['ticker', 'date', 'open', 'high', 'low', 'close', 'volume']])
```

	ticker	date	open	high	low	close	volume
0	SCON	2000-02-28	1395000.000	1797750.000	1368000.00	1779750.0	400.0
1	SCON	2000-02-29	2034000.000	2070000.000	1440000.00	1595250.0	400.0
2	SCON	2000-03-13	900000.000	1404000.000	861750.00	1363500.0	300.0
3	CODA	2001-02-05	1354500.000	1736700.000	1354500.00	1354500.0	34.0
4	TVIX	2013-02-26	1375000.000	1542500.000	1300000.00	1347500.0	100.0
5	SCON	2000-03-02	1120500.000	1368000.000	1098280.75	1261125.0	200.0
6	SCON	2000-02-25	873561.625	1369125.000	873000.00	1215000.0	200.0
7	TOPS	2016-08-01	783000.000	1276200.000	783000.00	1189800.0	100.0
8	SCON	2000-03-01	1368000.000	1512000.000	1107000.00	1134000.0	300.0
9	SCON	2000-03-14	1352250.000	1354500.000	1080000.00	1111500.0	100.0
10	TVIX	2013-03-19	997500.000	1110000.000	985000.00	1007500.0	100.0
11	SCON	2000-03-07	900000.000	1065936.625	859500.00	997875.0	100.0
12	TVIX	2013-06-24	942500.000	975000.000	895000.00	950000.0	100.0
13	TVIX	2013-06-20	810000.000	962500.000	800000.00	915000.0	100.0
14	SCON	2000-03-06	1222875.000	1222875.000	885375.00	895500.0	200.0

```
In [147... #Define decades_clean
for name in ['decades', 'decades_clean']:
    if name in globals():
        del globals()[name]
gc.collect()
```

Out[147... 14474

```
In [148... #light version
mask = ~stock_prices['adj_close'].between(0, 100_000)
print(stock_prices.loc[mask, 'decade'].value_counts().sort_index())
print(stock_prices.loc[mask, 'ticker'].value_counts().head(10))
```

```
decade
1980    461
1990    574
2000   2621
2010    403
Name: count, dtype: int64
ticker
YRCW    1152
ELJ     1100
AAN      824
MBVX     207
SAB      194
TVIX     182
CODA     155
CTIC     100
ABIO      63
SCON      44
Name: count, dtype: int64
```

In []:

In []:

In []: